SIMATS ENGINEERING

DEEP LEARNING LABORATORY

EXPERIMENT 6

GRADIENT DESCENT, SGD, MOMENTUM AND ADAM COMPARISON

Item	Details
Aim	To implement and compare Gradient Descent, Stochastic Gradient Descent (SGD), Momentum and Adam.
Tools	Google Colab, Python, NumPy, TensorFlow/Keras and Matplotlib
Dataset	Synthetic linear regression data: y = 3x + 2 + noise
Evaluation	Mean Squared Error (MSE), learned weight and bias

Aim and theory

The aim of this experiment is to observe how different optimization algorithms update the trainable parameters of a neural model and reduce prediction error. A single Dense neuron is used to learn a linear relationship from synthetic data. The model predicts ■ = wx + b, and the loss is Mean Squared Error (MSE) = (1/N) Σ(y■ – ■■)². During training, gradients indicate how the weight and bias should change to reduce the loss.

Gradient Descent calculates each update using the full dataset. SGD uses one example per update and may show a noisier loss curve. Momentum combines the current gradient with a fraction of the previous update, while Adam adapts each parameter's update using moving averages of gradients and squared gradients. All four methods are compared over 100 epochs.

Algorithm / Procedure

1. Import NumPy, TensorFlow and Matplotlib.
2. Generate 100 input points between –2 and 2 and create targets using y = 3x + 2 with small Gaussian noise.
3. Define a one-neuron Dense model and initialize its weight and bias to zero for a fair comparison.
4. Train separate models using Gradient Descent, SGD, Momentum and Adam.
5. Record the full-dataset MSE after every epoch.
6. Plot all loss histories on the same axes and compare the final MSE, learned weight and bias.

Important note about the submitted Colab screenshots

The uploaded screenshots show the dataset plot, but the optimizer comparison graph is blank and displays a legend warning. This usually means the results dictionary was empty when the plotting cell ran. Run the training cell (Cell 3) successfully before running the plotting cell (Cell 4). The comparison graph included in this report is a reproduced reference plot from the same dataset and optimizer update rules, not a screenshot captured from the user's Colab session.

Code — Cell 1: imports and dataset

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

np.random.seed(42)
tf.random.set_seed(42)

X = np.linspace(-2, 2, 100).reshape(-1, 1).astype(np.float32)
noise = np.random.normal(0, 0.2, (100, 1)).astype(np.float32)
y = 3 * X + 2 + noise

print("TensorFlow version:", tf.__version__)
print("Input shape:", X.shape)
print("Target shape:", y.shape)

plt.scatter(X, y, label="Training data")
plt.xlabel("X"); plt.ylabel("y")
plt.title("Synthetic Linear Regression Dataset")
plt.legend(); plt.grid(True); plt.show()
```

Code — Cell 2: model and optimizers

```
def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(1,)),
        tf.keras.layers.Dense(1)
    ])
    model.layers[0].set_weights([
        np.array([[0.0]], dtype=np.float32),
        np.array([0.0], dtype=np.float32)
    ])
    return model

optimizers = {
    "Gradient Descent": tf.keras.optimizers.SGD(learning_rate=0.05),
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.05)
}
print("Model and all 4 optimizers are ready!")
```

Code — Cell 3: training loop

```
results = {}
epochs = 100
batch_size = 16

for name, optimizer in optimizers.items():
    model = create_model()
    loss_history = []

    for epoch in range(epochs):
        if name == "Gradient Descent":
            current_batch_size = len(X)
        elif name == "SGD":
            current_batch_size = 1
        else:
            current_batch_size = batch_size

        indices = (np.arange(len(X)) if name == "Gradient Descent"
                   else np.random.permutation(len(X)))

        for start in range(0, len(X), current_batch_size):
            ids = indices[start:start + current_batch_size]
            xb, yb = X[ids], y[ids]

            with tf.GradientTape() as tape:
                pred = model(xb, training=True)
                loss = tf.reduce_mean(tf.square(yb - pred))

            grads = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(zip(grads, model.trainable_variables))

        mse = tf.reduce_mean(tf.square(y - model(X, training=False)))
        loss_history.append(float(mse.numpy()))

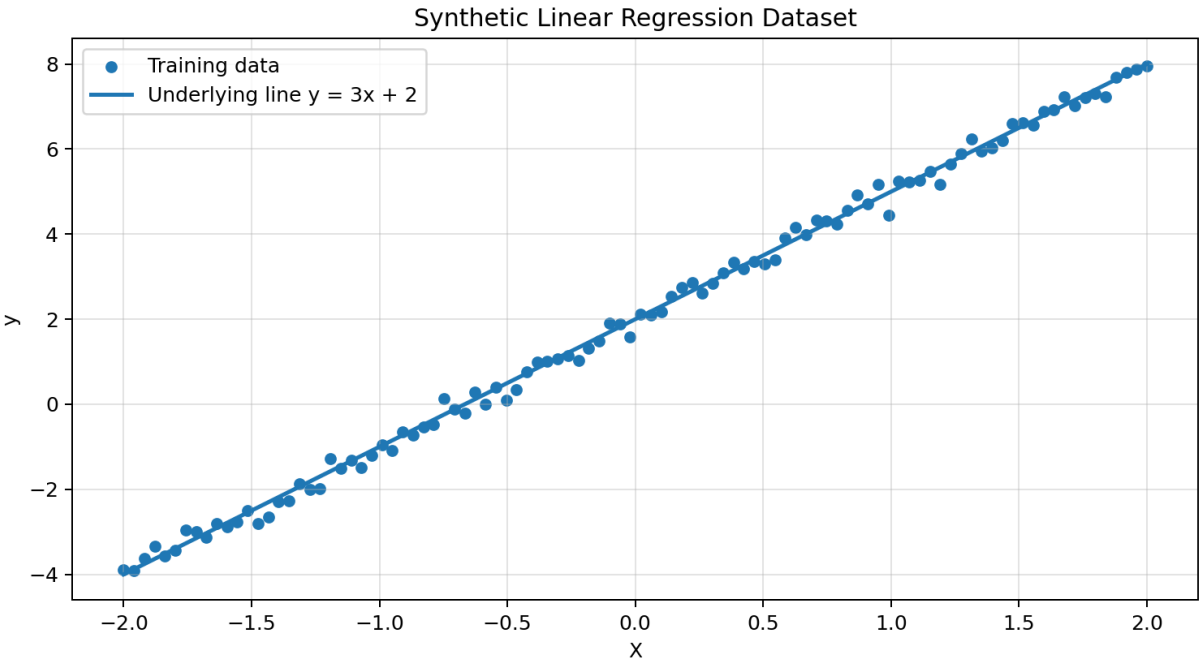
    results[name] = {
        "loss": loss_history,
        "weights": model.get_weights()
    }
    print(name, "Final MSE:", round(loss_history[-1], 4))
```

Code — Cell 4: optimizer comparison graph

```
plt.figure(figsize=(10, 6))
for name, result in results.items():
    plt.plot(result["loss"], label=name)

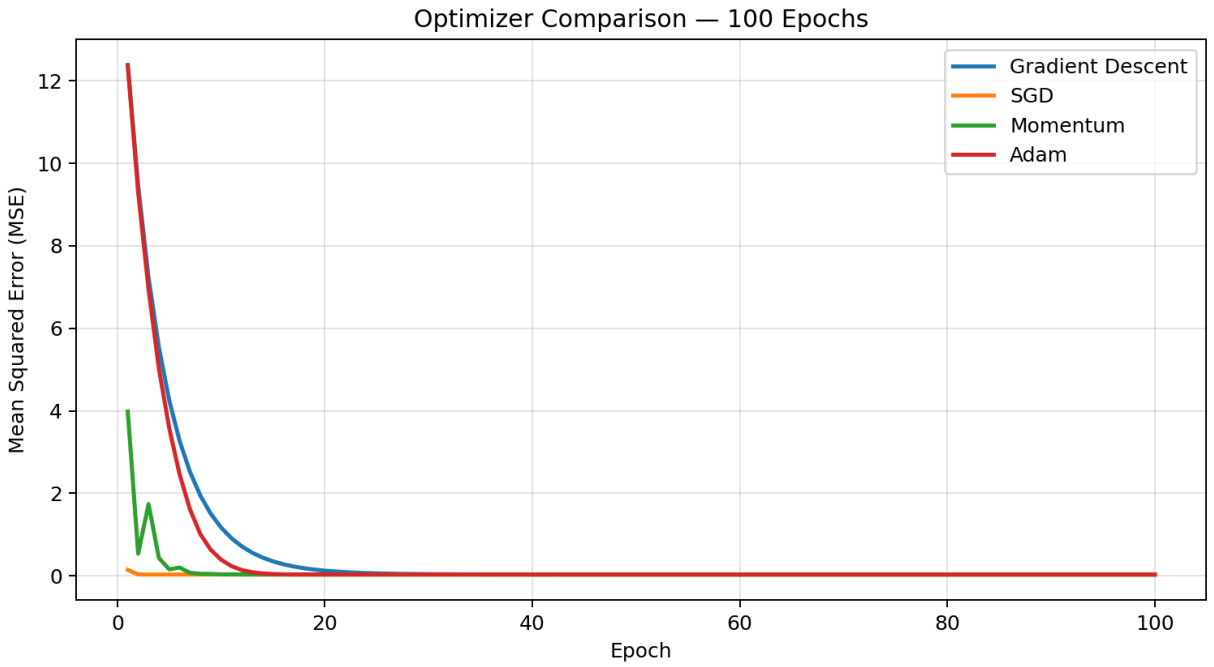
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error (MSE)")
plt.title("Optimizer Comparison")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output 1 — Synthetic dataset



The scatter points follow an approximately straight-line pattern. This is consistent with the synthetic target relationship y = 3x + 2 plus small random noise.

Output 2 — Optimizer comparison



Reference final metrics

Optimizer	Final MSE	Learned weight	Learned bias
Gradient Descent	0.032596	3.0069	1.9792
SGD	0.032897	3.0154	1.9650
Momentum	0.032630	3.0106	1.9832
Adam	0.032702	3.0144	1.9739

Interpretation

All four optimizers reduce the error and learn parameters close to the underlying values w = 3 and b = 2. The reference run gives final MSE values close to 0.033. Small differences between methods are expected because they use different batch sizes, update rules and stochastic orderings. The best optimizer should be judged from the loss curve and repeatability rather than from one value alone.

Observation

1. The training data have a strong linear relationship.
2. The optimizers learn a weight near 3 and bias near 2.
3. SGD can have a more variable trajectory because it updates one example at a time.
4. Momentum and Adam use gradient history to alter parameter updates.
5. Final MSE is low for all four methods on this simple dataset.

Result

Gradient Descent, SGD, Momentum and Adam were implemented and compared using a synthetic linear regression dataset. Their loss histories and final MSE values were examined. The reproduced reference results show that all four methods learn the underlying linear relationship with low error.

Before submitting

Run Cell 3 and confirm four optimizer results print. Then run Cell 4; the graph should show four labelled curves. If it remains blank, select Runtime > Run all and check Cell 3 for errors.